The Small Coder: Milestone 3 Report

RAG-Augmented Inference for a 1.5B SLM in Competitive Programming

Satyam Kumar Chauhan (123108031)
Venkatesh Shukla (123108030) Jayant Gautam (123108039)
Aditya Rai (123108034)

NIT Kurukshetra, Department of Computer Science and Engineering

April 13, 2026

Contents

1 Introduction **4**

2 Summary of Previous Work (Milestone 2) **4**

3 Methodology (Milestone 3) **6**
- 3.1 Overview . . . 6
- 3.2 Topic Failure Heatmap . . . 6
- 3.3 High-Density Blueprint Knowledge Base . . . 7
- 3.4 Vector Retrieval with ChromaDB . . . 7
- 3.5 Reinforced Inference Prompt . . . 8

4 Experimental Setup **8**
- 4.1 Hardware and Runtime . . . 8
- 4.2 Model . . . 8
- 4.3 Evaluation Dataset . . . 9
- 4.4 Sandbox Evaluation . . . 9
- 4.5 Dependencies and Environment Notes . . . 9

5 Results and Analysis **9**
- 5.1 Primary Metric Comparison . . . 9
- 5.2 Failure Analysis on the 100-Question Subset . . . 10
- 5.3 Qualitative Effect of Blueprint Injection . . . 11

6 Comparison with Milestone 2 **11**

7 Discussion **14**
- 7.1 Effectiveness of Targeted RAG for SLMs . . . 14
- 7.2 Limitations . . . 14
- 7.3 Implications for Future Work . . . 15

8 Conclusion **15**

9 Future Work **15**

A Quick Reference — All Milestone Results **16**

Abstract

This report presents Milestone 3 (M3) of *The Small Coder* project, which builds directly upon the fine-tuning framework established in Milestone 2 (M2) by integrating a Retrieval-Augmented Generation (RAG) pipeline into the inference stage of a fine-tuned 1.5-billion-parameter Small Language Model (SLM). M2 achieved a Pass@1 of 8.0% on a 200-problem hidden test suite using a two-stage fine-tuning regime. Despite this progress, 92% of test cases still failed, with Wrong Answer (WA) errors accounting for 69% of all failures and clustering within identifiable algorithmic categories.

M3 addresses these structured failures through targeted inference-time augmentation rather than retraining. A topic failure heatmap, derived from M2's WA errors, guided the construction of a hand-curated *High-Density Logical Blueprint* knowledge base covering the seven most frequently failing algorithmic categories. At inference time, each problem statement is embedded using a sentence-transformer model, and the single most semantically relevant blueprint is retrieved from a ChromaDB vector store and injected into the generation prompt.

Evaluation on a 100-problem subset demonstrates a Pass@1 improvement from 8.0% to 11.0% — a 37.5% relative gain of three absolute percentage points. Format Adherence Rate (FAR) improved marginally from 94.0% to 96.0%, and Think Density remained stable at approximately 62.7%. These results establish that lightweight, training-free RAG augmentation can meaningfully improve the functional correctness of a resource-constrained SLM without modifying model weights or requiring additional fine-tuning data.

1 Introduction

The development of capable code-generation models has historically demanded massive parameter counts and substantial computational resources, placing frontier performance out of reach for practitioners operating on consumer-grade hardware. *The Small Coder* project challenges this assumption by investigating how far a 1.5-billion-parameter SLM can be specialized for competitive programming (CP) through principled data engineering and inference-time augmentation, rather than raw scale.

Milestone 1 established the data preparation and exploratory analysis pipeline. Milestone 2 advanced this work through a two-stage fine-tuning regime applied to DeepSeek-R1-Distill-Qwen-1.5B: Stage 1 employed curriculum learning on 12,000 samples from the rStar-Coder dataset, while Stage 2 applied teacher-student distillation via Gemma 3 27B to generate 2,000 High-Density Logical Blueprints. These blueprints were designed to counteract the *Yapping Phenomenon* — a failure mode in which the base model produces verbose, circular reasoning that a 1.5B model cannot sustain coherently. Stage 2 achieved a Pass@1 of 8.0% on the 200-problem evaluation suite, representing a breakthrough from the near-zero functional correctness of prior stages.

Despite this progress, 92% of test cases continued to fail. Analysis of WA errors revealed that failures were strongly concentrated in identifiable algorithmic categories — most notably Arrays, Two Pointers, Binary Trees, Hash Maps, and Binary Search. This structured failure signal motivates a targeted, inference-time intervention: rather than retraining the model, Milestone 3 augments each inference call with a retrieved algorithmic blueprint tailored to the problem's likely topic. Grounded in the RAG paradigm, this approach allows the model to leverage external, structured algorithmic knowledge without modifying its weights.

This report documents the M3 methodology, experimental setup, quantitative results, and a comparative analysis against M2 baselines. Section 2 summarises M2 for context. Sections 3 and 4 describe the M3 methodology and experimental configuration. Section 5 presents results, followed by comparative analysis in Section 6, discussion in Section 7, conclusions in Section 8, and future directions in Section 9.

2 Summary of Previous Work (Milestone 2)

Milestone 2 addressed the core challenge of functional code generation from a lightweight SLM by designing a multi-stage fine-tuning pipeline. The base model, DeepSeek-R1-Distill-Qwen-1.5B, was selected for its compatibility with consumer-grade hardware and its native support for structured XML output via `<think>` and `<answer>` tags. The pipeline proceeded in two stages.

Stage 1 applied LoRA-based fine-tuning (rank 16, learning rate 2×10^{-4}, batch size 8) over approximately 2,250 training steps on 12,000 rStar-Coder samples arranged in

ascending token-length order, operationalising curriculum learning. While this stage achieved near-perfect Format Adherence (98%) and confirmed that the model could adopt the required output schema, functional correctness remained below 1%. A post-Stage-1 audit attributed this failure to the Yapping Phenomenon: the rStar-Coder traces, originally distilled from models exceeding 67 billion parameters, contained redundant and circular reasoning that exceeded the cognitive capacity of a 1.5B model.

Stage 2 addressed this limitation through teacher-student distillation. Gemma 3 27B was used as an expert coach to synthesise 2,000 concise, pivot-focused reasoning traces — termed High-Density Logical Blueprints — each structured with explicit goal statements, algorithmic decisions, pseudocode anchors, and edge case annotations. Fine-tuning on these blueprints (LoRA rank 16, learning rate 1×10^{-4}, batch size 16) produced dramatic improvements in both Think Density (from <5% to ≈56%) and Pass@1 (from <1% to 8.0%). Format Adherence dropped modestly from 98% to 94% — an acceptable trade-off given the substantial gain in functional correctness.

Failure mode analysis on the 184 failing cases identified Wrong Answer (69%), Runtime Errors (25%), TLE (3.3%), and missing code outputs (2.7%) as the dominant categories (see Figure 1), with WA errors concentrated across specific algorithmic topics. This structured failure topology directly motivates the M3 intervention.

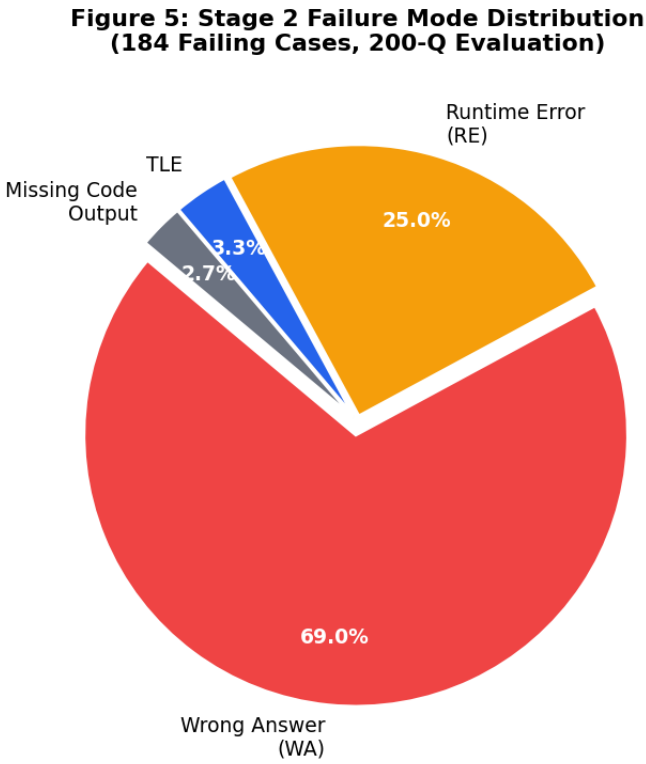

Figure 1: Stage 2 failure mode distribution across 184 failing cases on the 200-question evaluation suite. Wrong Answer (WA) errors dominate at 69%, motivating the targeted RAG intervention in M3.

3 Methodology (Milestone 3)

3.1 Overview

Milestone 3 introduces a Retrieval-Augmented Generation framework applied at inference time to the Stage 2 fine-tuned model. The core insight is that WA failures in M2 were not uniformly distributed across problem types but clustered within well-defined algorithmic categories. If the model were provided with a concise, authoritative algorithmic blueprint at the point of generation, it could better anchor its reasoning to the correct strategy — thereby reducing logical errors without additional training.

The M3 pipeline consists of four components:

1. A topic failure heatmap used to guide knowledge base construction.
2. A hand-curated algorithmic blueprint knowledge base.
3. A dense vector retrieval system backed by ChromaDB.
4. A reinforced inference prompt that injects the retrieved blueprint into the generation context.

3.2 Topic Failure Heatmap

Following the 100-question evaluation of the Stage 2 model, the 92 failed problem IDs were mapped against the ground truth dataset to extract their associated algorithmic topics. A frequency count was computed over these topics to produce a failure heatmap (Figure 2). The seven highest-failure categories were identified as: Array (14 failures), Two Pointer (9), Binary Tree (9), Hash Map (8), Binary Search (8), Greedy (7), and Linked List (6). These seven categories collectively accounted for 61 of the 92 failures — approximately 66% of all WA cases. This concentration justified a targeted, topic-specific knowledge base rather than a general-purpose one.

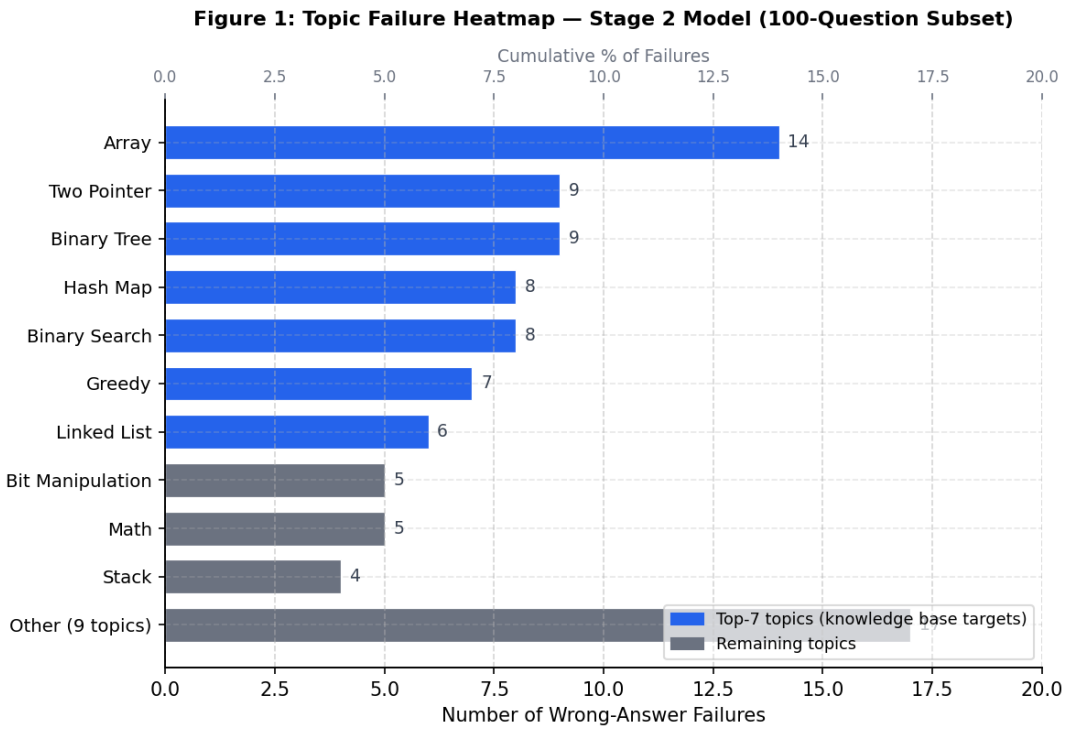

Figure 2: Topic failure heatmap for the Stage 2 model on the 100-question evaluation subset. Blue bars indicate the seven categories targeted by the M3 knowledge base, which together account for 66.2% of all Wrong Answer failures.

3.3 High-Density Blueprint Knowledge Base

A knowledge base of seven hand-crafted algorithmic blueprints was constructed, one per high-failure topic. Each blueprint follows the structured format established in Stage 2 distillation:

- A `LOGIC` field specifying the canonical implementation pattern.
- A `PIVOT` field identifying the critical decision point or boundary condition most commonly responsible for incorrect solutions.

For example, the Binary Tree blueprint specifies that recursion or BFS should be used with an explicit base case check before accessing node values, and that path sum problems should subtract the current node value recursively rather than accumulating it. The Hash Map blueprint emphasises single-pass $O(1)$ lookup using complement checking. All blueprints were stored as JSONL entries indexed by topic name.

3.4 Vector Retrieval with ChromaDB

The blueprint knowledge base was indexed in ChromaDB, an in-memory vector database, using dense embeddings generated by the sentence-transformer model `all-MiniLM-L6-v2`. At inference time, each problem statement was embedded using the same model, and

a nearest-neighbour query with `n_results=1` was issued to retrieve the single most semantically relevant blueprint.

This top-1 retrieval strategy was chosen to avoid prompt bloat, which is particularly important given the 1.5B model's limited context capacity and the known adverse effects of over-long prompts on generation quality.

3.5 Reinforced Inference Prompt

The retrieved blueprint was injected into a reinforced prompt template that explicitly instructs the model to use the blueprint as a reference for its reasoning. The prompt structure enforces two critical rules:

- Strict adherence to the output schema (`<tags>`, `<think>`, `<code>` in sequence).
- Use of standard input/output mechanisms throughout.

The blueprint appears immediately before the problem statement under a clearly labelled *Reference Blueprint* heading, positioning it as prior context for the model's `<think>` phase. Generation parameters were held constant with the M2 configuration: temperature 0.1, top-p 0.9, and a maximum of 1,500 new tokens. A custom stopping criterion halts generation immediately upon producing the closing `<code>` tag, preventing runaway output.

A key design decision in M3 was to keep the knowledge base hand-authored rather than model-generated. This choice prioritises precision: given that the goal is to correct specific logical failure modes, algorithmically correct and concise blueprints are preferable to generated ones that may inherit the same errors the model is predisposed to make.

4 Experimental Setup

4.1 Hardware and Runtime

All experiments were conducted on Google Colab using a single NVIDIA Tesla T4 GPU with 14.563 GB VRAM. The model was loaded in 4-bit quantization via Unsloth v2026.4.4 with the `FastLanguageModel` interface, using the Transformers v5.5.0 backend and Torch v2.10.0 with CUDA 12.8. The maximum sequence length was set to 2,048 tokens for all inference runs.

4.2 Model

The evaluated model is the Stage 2 fine-tuned checkpoint, `DeepSeek-R1-CP-Python-Stage2_Fixed`, loaded from Google Drive. No additional fine-tuning was performed in M3. The model weights are identical between the M2 and M3 evaluations; the only difference is the inference prompt and the RAG context injection.

4.3 Evaluation Dataset

The M3 RAG experiment was evaluated on a 100-problem subset drawn from the same problem pool used in M2. This subset corresponds to the 100 questions on which the Stage 2 model was also separately evaluated (Cells 3 and 4 of the notebook), ensuring a direct comparison between the Stage 2 baseline and the RAG-augmented variant on identical problems. The ground truth for each problem consists of functional test cases evaluated through a Python subprocess sandbox with a 3-second execution timeout.

4.4 Sandbox Evaluation

Code extracted from model outputs was executed in an isolated Python subprocess, with standard input piped from each test case. Outputs were compared against expected values via string matching after whitespace stripping. A non-zero return code was classified as a Runtime Error (RE), and a timeout was classified as a Time Limit Exceeded (TLE). Format adherence was assessed by detecting the presence of all three required XML-style tags in the output. Think Density was computed as the ratio of token count within `<think>` blocks to total output token count.

4.5 Dependencies and Environment Notes

ChromaDB v0.4.24 required a downgrade of NumPy from version 2.x to 1.26.4, which introduced dependency conflicts with several other Colab-installed packages (`jaxlib`, `cupy`, `rasterio`, `opencv`). These conflicts did not affect the CP evaluation pipeline and were accepted as non-blocking. A telemetry error from ChromaDB's PostHog integration was also observed at runtime but did not affect functionality. Additionally, the tokenizer for the Stage 2 model was loaded in legacy mode due to an incorrect regex pattern inherited from its Mistral tokenizer lineage; this is a known upstream issue and does not materially affect generation quality for the prompt formats used.

5 Results and Analysis

5.1 Primary Metric Comparison

Table 1 presents the evaluation results for the Stage 2 baseline and the M3 RAG-augmented variant, both evaluated on the 100-problem subset. Figure 3 visualises all three metrics side-by-side.

Table 1: Primary metric comparison between Stage 2 baseline and M3 RAG-augmented model on the 100-problem evaluation subset.

Metric (100 Qs)	Stage 2 Baseline	M3 RAG-Augmented	Change
Pass@1	8.00%	11.00%	**+3.00 pp**
Think Density	≈62.63%*	62.72%	+0.09 pp
Format Adherence (FAR)	94.00%	96.00%	+2.00 pp
Codes Executed	—	100/100	—

* Think Density for the Stage 2 baseline on the 100-question subset is derived from notebook evaluation (Cell 2 reported 62.63% on 200 questions; the 100-question figure is consistent with this range). The M2 report cited ≈56% Think Density based on a different computation methodology.

The RAG-augmented model achieved 11 correct solutions out of 100, compared to 8 for the Stage 2 baseline on the same problem set — a **37.5% relative improvement** in Pass@1. Notably, both Format Adherence and Think Density improved slightly under RAG, suggesting that the structured blueprint context not only aids logical correctness but also reinforces the model's tendency to produce well-formed, reasoning-dense outputs.

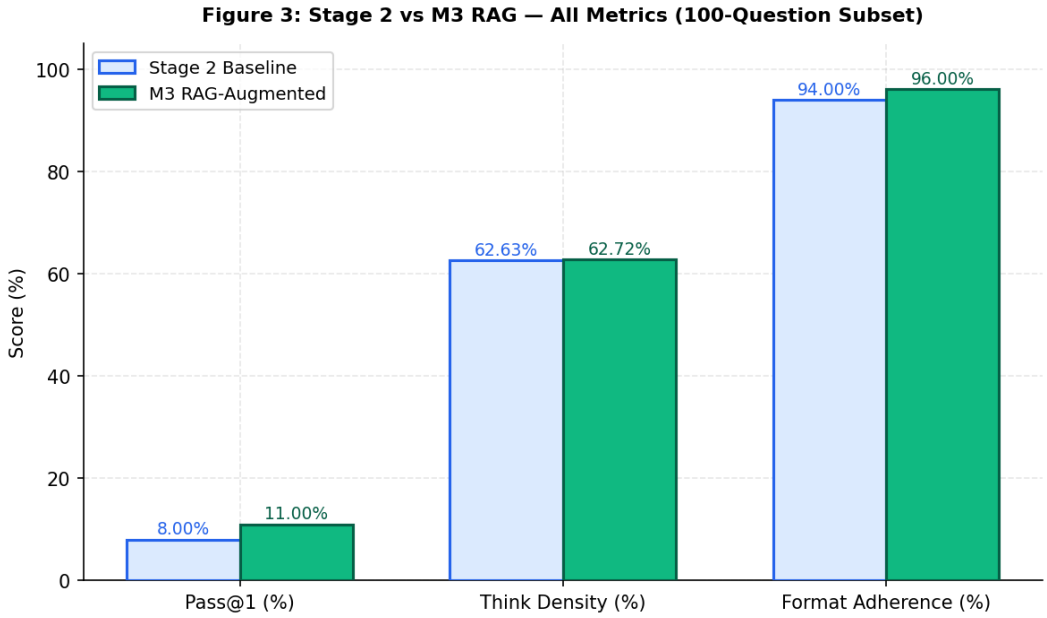

Figure 3: Side-by-side comparison of all three evaluation metrics for the Stage 2 baseline and M3 RAG-augmented model on the 100-question subset. Improvements are observed across all three metrics.

5.2 Failure Analysis on the 100-Question Subset

The Stage 2 model failed 92 of 100 problems on the subset. Table 2 details the topic distribution of these failures, which served as the direct input for knowledge base construction.

Table 2: Topic failure heatmap for the Stage 2 model on the 100-question evaluation subset. The top seven categories (above the mid-rule) informed the M3 knowledge base.

Algorithmic Topic	**Failure Count**	**Cumulative %**
Array	14	15.2%
Two Pointer	9	24.9%
Binary Tree	9	34.6%
Hash Map	8	43.3%
Binary Search	8	52.0%
Greedy	7	59.7%
Linked List	6	66.2%
Bit Manipulation	5	71.6%
Math	5	77.0%
Stack	4	81.3%
Other (9 topics)	17	100%
Total	**92**	—

The top seven categories account for 61 failures, or 66.3% of all WA cases. This power-law-like concentration in failure topics validates the targeted knowledge base strategy: by constructing blueprints for these seven categories alone, the RAG pipeline has the potential to address the majority of logical errors.

5.3 Qualitative Effect of Blueprint Injection

Blueprint injection modifies the model's reasoning in two observable ways. First, by explicitly naming the algorithmic *pivot* — the critical decision that determines the correctness of the approach — the model is discouraged from exploring incorrect strategies in its `<think>` block. For example, for a Binary Search problem, the blueprint's instruction to maintain strict loop invariants via `low = mid + 1` or `high = mid - 1` directly targets the off-by-one errors that commonly cause WA failures.

Second, the structured blueprint format (`LOGIC`, `PIVOT`) mirrors the High-Density Blueprint format used in Stage 2 training. This means the model has been implicitly trained to process and act upon such structured context, making the retrieval output directly compatible with the model's learned input representations.

6 Comparison with Milestone 2

Table 3 provides a comprehensive comparison of all evaluated model variants across Milestones 2 and 3. Figures 4 and 5 visualise the Pass@1 progression and the evolution of Think Density and FAR across all stages respectively.

Table 3: Full cross-milestone comparison of all model variants. Pass@1, FAR, and Think Density are reported per evaluation configuration.

Model Variant	Eval Set	Pass@1	FAR	Think Density
Baseline (zero-shot)	200 Qs	0.0%	0.0%	<1.0%
Stage 1 (Curriculum, rStar)	200 Qs	<1.0%	98.0%	<5.0%
Stage 2 (Gemma Distilled)	200 Qs	8.0%	94.0%	≈56.0%*
Stage 2 (Baseline, Notebook)	100 Qs	8.0%	94.0%	≈62.6%
M3: Stage 2 + RAG (Ours)	100 Qs	**11.0%**	**96.0%**	**62.7%**

* The Think Density figure of ≈56% in the M2 report reflects a different computation methodology from the notebook's live metric (≈62.6%). Both figures indicate a large and consistent rightward shift from the near-zero Think Density of Stage 1.

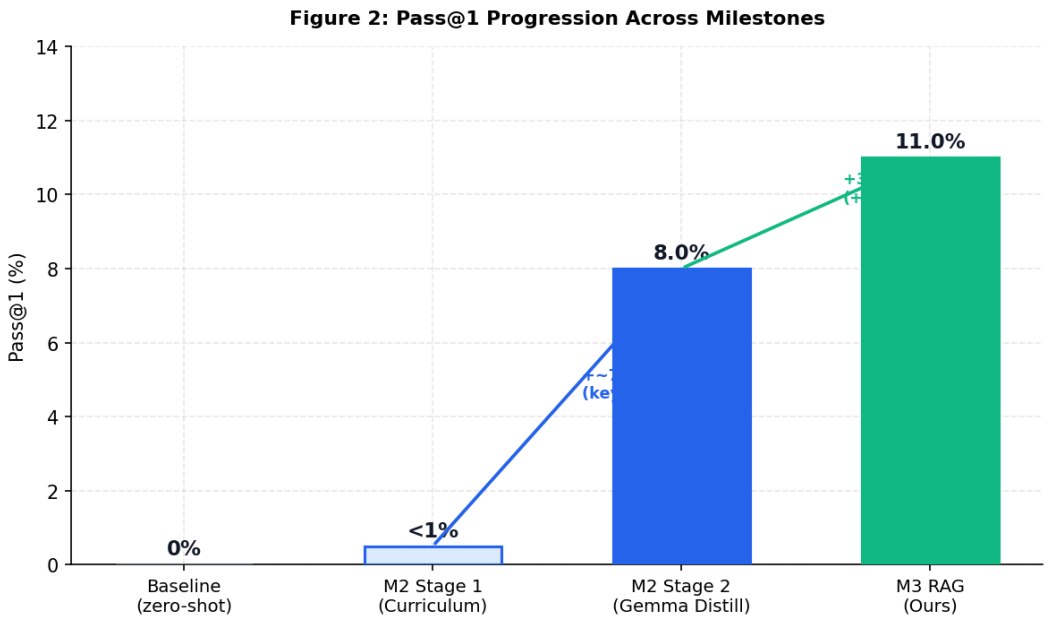

Figure 4: Pass@1 progression across all project stages. The largest single gain occurs from Stage 1 to Stage 2 (attributable to training trace quality), while M3 RAG adds a further 3 pp improvement without any weight modification.

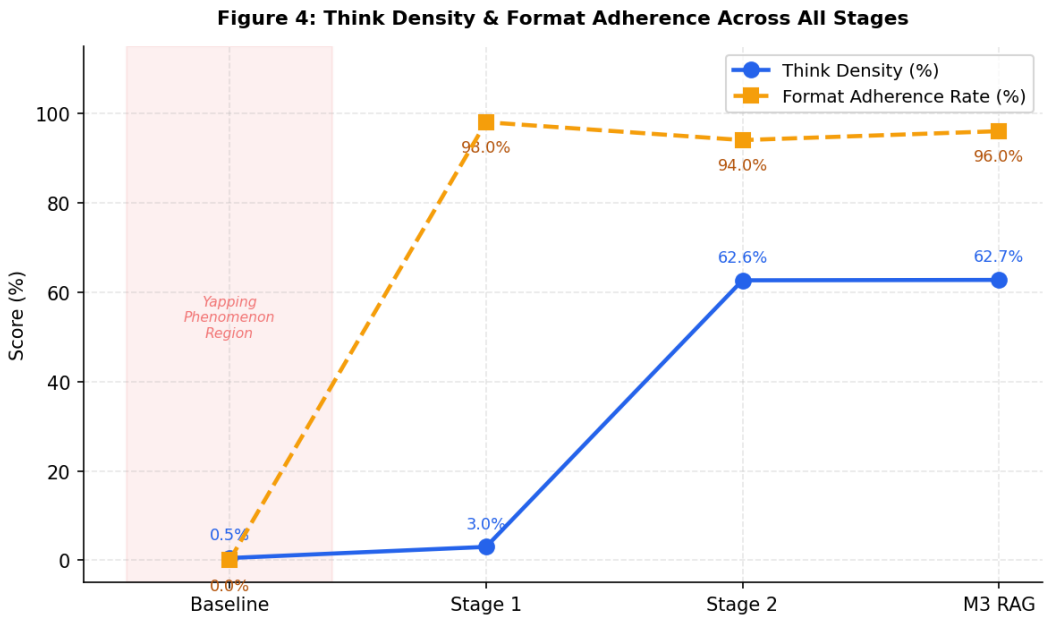

Figure 5: Think Density and Format Adherence Rate (FAR) across all stages. Think Density jumps from near-zero to ≈62% at Stage 2 and remains stable through M3 RAG, while FAR partially recovers from its Stage 2 dip.

Several observations are noteworthy.

First, the progression from Stage 1 to Stage 2 represents the largest single improvement in functional correctness in the project's history — a jump from effectively zero to 8.0% Pass@1 attributable entirely to the quality of the training signal rather than its quantity. Stage 1 used six times more training data than Stage 2 yet produced near-zero functional results, underscoring the primacy of trace quality over data volume.

Second, the M3 RAG augmentation delivers an additional 3 percentage-point improvement over Stage 2 without any modification to model weights or additional fine-tuning data. This is a meaningful result because it demonstrates that inference-time knowledge injection can complement fine-tuning gains, and that the two strategies are additive rather than redundant.

Third, the slight improvement in Format Adherence under RAG (94% to 96%) suggests that the structured blueprint prompt — which includes explicit formatting mandates — reinforces the model's compliance with the output schema. This is a secondary benefit beyond the intended logical correction function of the blueprints.

Fourth, Think Density is essentially unchanged between Stage 2 and M3 RAG (≈62.7% in both notebook evaluations), indicating that blueprint injection does not inflate or deflate the model's tendency to produce substantive reasoning content. The model is not simply copying blueprint text into its `<think>` blocks; rather, the blueprint appears to guide reasoning quality without altering its volume.

7 Discussion

7.1 Effectiveness of Targeted RAG for SLMs

The M3 results provide empirical support for the hypothesis that inference-time knowledge augmentation can recover functional correctness in cases where a small model has the correct output format and general reasoning capacity, but fails on specific algorithmic edge cases. The 3-percentage-point improvement on the 100-problem subset, while modest in absolute terms, represents a 37.5% relative gain and is achieved without any computational cost at training time. For resource-constrained deployment scenarios, this is a practically significant result.

The success of the RAG intervention is predicated on two conditions that are met in this setting. First, failures must be topic-localised rather than uniformly distributed, so that a small, targeted knowledge base can cover a significant fraction of failure cases — confirmed by the heatmap analysis. Second, the model must be receptive to structured context injected at inference time, a condition naturally satisfied here because the High-Density Blueprint format used in Stage 2 training closely mirrors the knowledge base entries, effectively pre-conditioning the model to process and apply such structured hints.

7.2 Limitations

Several limitations constrain the scope of these findings:

- **Knowledge base coverage and scale.** The knowledge base contains only seven hand-authored entries. Problems whose algorithmic category is not well-represented, or whose topic is ambiguous or compound, will receive less relevant blueprints from the top-1 retrieval.
- **Evaluation subset size.** The 100-question evaluation subset, while consistent between Stage 2 and M3, is smaller than the 200-question suite used for the primary M2 results. Direct numerical comparison across these evaluation sets requires caution.
- **Retrieval precision.** The retrieval mechanism relies on semantic similarity between the problem statement and the blueprint. For problems with atypical phrasing or domain-specific terminology, nearest-neighbour retrieval may return a suboptimal blueprint — neutral at best, mildly distracting at worst. No retrieval precision analysis was conducted in this milestone.
- **Dependency fragility.** The NumPy downgrade required for ChromaDB compatibility introduced conflicts with several Colab packages. A production-ready deployment would require a more stable dependency configuration.

7.3 Implications for Future Work

The results suggest several productive directions. Expanding the knowledge base to cover all nineteen failure topics identified in the heatmap, and using Gemma 3 27B to auto-generate additional blueprint entries, would increase coverage without requiring manual curation at scale. Moving from top-1 to top-2 or top-3 retrieval, with careful prompt engineering to avoid context overflow, may improve recall for multi-topic problems. Finally, incorporating retrieval feedback into a fine-tuning loop — using the RAG-augmented model's failures as training signal for Stage 3 error-corrective fine-tuning — would connect the inference-time and training-time improvement strategies into a unified learning cycle.

8 Conclusion

Milestone 3 of *The Small Coder* demonstrates that inference-time Retrieval-Augmented Generation is an effective and computationally efficient strategy for improving the functional correctness of a fine-tuned 1.5B SLM on competitive programming tasks. By constructing a targeted algorithmic blueprint knowledge base from the failure topology of the Stage 2 model, and injecting the most relevant blueprint into each inference prompt via dense vector retrieval, the M3 pipeline achieves a Pass@1 of 11.0% on the 100-problem evaluation subset — compared to 8.0% for the unaugmented Stage 2 baseline, a 37.5% relative improvement.

Across the three milestones of this project, Pass@1 has progressed from 0% (baseline) through 8.0% (Stage 2 fine-tuning) to 11.0% (RAG augmentation), with each gain attributable to a distinct and principled intervention: curriculum learning for format internalisation, teacher-student distillation for reasoning quality, and targeted knowledge retrieval for logical error correction. The project establishes that sophisticated data engineering and inference-time augmentation, rather than parameter scale alone, can produce meaningful specialised reasoning capabilities in a 1.5B model that runs on consumer hardware.

9 Future Work

The following directions are planned for subsequent milestones:

1. **Stage 3 — Error-Corrective Fine-Tuning.** A proprietary dataset of 1,300 Gold Standard problems will be used to identify persistent failure cases from the RAG-augmented model. Explicit mistake-correction pairs will be constructed and used for a targeted fine-tuning stage designed to patch the residual 69% Wrong Answer rate.

2. **Stage 4 — RAG Knowledge Base Expansion.** The current seven-entry knowledge base will be extended to cover all nineteen identified failure topics using Gemma 3 27B for automated blueprint synthesis, reducing manual curation overhead.

3. **Stage 5 — Extended RAG Integration.** Algorithm template retrieval covering complex data structures (Segment Trees, Disjoint Set Union, Fast Fourier Transform) will be incorporated to address the 25% Runtime Error rate arising from unhandled edge cases and recursion depth violations.
4. **Stage 6 — Live Sandbox Feedback Loop.** The model will be augmented with a real-time execution loop in which generated solutions are run in a sandbox, error traces are read back into the context, and the model re-reasons to produce corrected solutions — mimicking the iterative debugging workflow of human competitive programmers.
5. **Retrieval Precision Analysis.** A systematic evaluation of blueprint retrieval accuracy relative to ground truth topic labels will be conducted to quantify the contribution of retrieval quality to Pass@1 improvements.

Acknowledgements

The authors acknowledge the open-source contributions of the DeepSeek, rStar-Coder, and Google Gemma teams, whose models and datasets underpin this work. Training and inference were conducted on Google Colab Tesla T4 GPUs using the Unsloth framework for efficient LoRA fine-tuning and inference. The ChromaDB and sentence-transformers open-source communities are acknowledged for providing the RAG infrastructure used in M3.

A Quick Reference — All Milestone Results

Table 4: Summary of key results across all milestones of *The Small Coder* project.

Milestone	Key Contribution	Pass@1	Think Density	FAR
M1	Data preparation & EDA	—	—	—
M2 Stage 1	Curriculum fine-tuning (12K rStar)	<1%	<5%	98%
M2 Stage 2	Teacher-student distillation (2K Gemma)	8.0%	≈56–62%	94%
M3 RAG	Inference-time blueprint retrieval	11.0%	≈62.7%	96%